

A PROGRAMMER'S GUIDE

LEXICO

Language Documentation

GUIDE & REFERENCE

by

HAYTHEM GHARBI

*Readable words. Explicit types.
Deterministic programs.*

VERSION 1.0 · 2026

ABOUT THIS MANUAL



LEXICO is a compiled, word-based programming language designed to make common programming intent visible in the source. It uses readable words such as `plus`, `divided by`, `is greater than`, and `giveback` instead of relying only on symbolic operators.

This manual is about *writing* LEXICO programs. It describes the syntax implemented by the current repository, teaching declarations, expressions, conditions, loops, routines, collections, files, modules, and classes. It does not attempt to document the compiler's internal implementation.

The chapters are arranged in five parts. Part I introduces the language and how to run it. Part II covers its core: variables, types, operators, text, input and output, conditions, and loops. Part III shows how programs are built from routines, meshes, files, modules, and classes. Part IV gathers errors, complete programs, and recipes, and Part V is the reference and cheat sheet.

CONTENTS

I	BEGINNINGS	1
1	Welcome	2
1.1	What is LEXICO?	2
1.2	A first program	2
1.3	Why use LEXICO?	3
2	Getting started	4
2.1	Requirements	4
2.2	Build with CMake	4
2.3	Run a program	5
2.4	Your first complete program	5
3	Language basics	6
3.1	Statements	6
3.2	Comments	6
3.3	Whitespace and block markers	6
3.4	Identifiers and keywords	7
3.5	Case sensitivity	7
3.6	Program structure	7
II	THE LANGUAGE	8
4	Variables and assignment	9
4.1	Declare and initialize one variable	9
4.2	Declare several variables	9
4.3	Default initialization	9
4.4	Assignment	10
4.5	Compound assignment shortcut	10
4.6	Scope rules	10
5	Data types and literals	11
5.1	Type overview	11
5.2	Integers	11
5.3	Floating-point values	11
5.4	Characters	12
5.5	Strings	12
5.6	Booleans	12
6	Operators and expressions	13
6.1	Arithmetic operators	13
6.2	Precedence	13

6.3	<i>Unary operators</i>	14
6.4	<i>Comparisons</i>	14
6.5	<i>Logical operators</i>	14
7	Strings and text	15
7.1	<i>Concatenation</i>	15
7.2	<i>Remove a substring</i>	15
7.3	<i>Index a string</i>	15
7.4	<i>Length</i>	16
7.5	<i>Extraction</i>	16
8	Input and output	17
8.1	<i>Print values</i>	17
8.2	<i>Read scalar input</i>	17
8.3	<i>Message expressions</i>	17
8.4	<i>Boolean input</i>	18
9	Conditions	19
9.1	<i>Simple if</i>	19
9.2	<i>otherwise</i>	19
9.3	<i>otherwise when</i>	19
9.4	<i>Nested conditions</i>	20
10	Loops	21
10.1	<i>while</i>	21
10.2	<i>Numeric for</i>	21
10.3	<i>Collection-bound for</i>	22
10.4	<i>repeat until</i>	22
10.5	<i>attempt up to</i>	22
10.6	<i>Matrix loops</i>	22
III	BUILDING PROGRAMS	24
11	Routines	25
11.1	<i>Define a routine</i>	25
11.2	<i>Call a routine and use its result</i>	25
11.3	<i>Discard a result with <code>emit</code></i>	25
11.4	<i>No-argument calls</i>	26
11.5	<i>Parameters and validation</i>	26
11.6	<i>Recursion</i>	26
12	Meshes, arrays, tables, and matrices	27
12.1	<i>Typed mesh with a size</i>	27
12.2	<i>Dynamic mesh</i>	27
12.3	<i>Initialize from values</i>	27
12.4	<i>Legacy explicit array/table forms</i>	28
12.5	<i>Read and write elements</i>	28
12.6	<i>Append and sort</i>	28
12.7	<i>Count, membership, and position</i>	28

12.8	<i>Matrices</i>	29
13	File operations	30
13.1	<i>Open or create a file</i>	30
13.2	<i>Write lines</i>	30
13.3	<i>Read lines</i>	30
13.4	<i>Clear, rename, and close</i>	31
13.5	<i>Complete file example</i>	31
14	Modules	32
15	Classes and profiles	33
15.1	<i>Define a class</i>	33
15.2	<i>Inheritance and override</i>	33
15.3	<i>Instantiate a profile</i>	34
15.4	<i>Class routine with a return type</i>	34
16	Conversions and built-in expressions	35
16.1	<i>Statement conversion</i>	35
16.2	<i>Expression conversion</i>	35
16.3	<i>Built-in expressions</i>	35
IV	PRACTICE	37
17	Errors and limitations	38
18	Complete programs	40
18.1	<i>Beginner: total calculator</i>	40
18.2	<i>Intermediate: grade classification</i>	40
18.3	<i>Intermediate: routine and loop</i>	41
18.4	<i>Advanced: mesh search</i>	41
18.5	<i>Advanced: file-backed notes</i>	41
19	Recipes	42
19.1	<i>Factorial with a loop</i>	42
19.2	<i>Find a maximum in a mesh</i>	42
19.3	<i>Sort a typed mesh</i>	42
19.4	<i>Validate a bounded retry</i>	43
19.5	<i>Convert text to a number</i>	43
V	REFERENCE	44
20	Language reference	45
20.1	<i>Keywords</i>	45
20.2	<i>Statement forms</i>	45
20.3	<i>Control-flow forms</i>	45
20.4	<i>Routine forms</i>	46
20.5	<i>Collection forms</i>	46
20.6	<i>File forms</i>	46
21	Cheat sheet	48

PART

I

BEGINNINGS



CHAPTER

1

WELCOME

1.1 *What is LEXICO?*

LEXICO is a statically typed language with:

- ◆ readable, keyword-based expressions;
- ◆ indentation-sensitive blocks using > markers;
- ◆ integers, floating-point values, characters, strings, and booleans;
- ◆ conditions and four loop forms;
- ◆ typed routines with return values;
- ◆ arrays, tables, meshes, and matrices;
- ◆ file blocks and module loading;
- ◆ classes with inheritance, profiles, and routine overrides.

LEXICO is a **controlled language**, not unrestricted natural language. The words make programs easier to read, but the accepted forms are still precise.

1.2 *A first program*

```
create int price set value 20;
create int quantity set value 3;
create int total set value price times quantity;

print "Total:", total;
```

The program:

1. declares price and initializes it to 20;
2. declares quantity and initializes it to 3;
3. calculates price times quantity and stores the result in total;
4. prints two values separated by a space.

A LEXICO statement normally ends with `;`. A block header, such as `if ... then`, is followed by indented lines rather than a semicolon.

1.3 *Why use LEXICO?*

LEXICO is useful when you want source code that reads close to a plain-language description while retaining explicit types and deterministic behavior. Its most distinctive rules are:

- ♦ arithmetic operators are words;
- ♦ comparisons are phrases;
- ♦ blocks are marked by one or more `>` characters;
- ♦ routines use `giveback`, `emit`, and `outcome of`;
- ♦ collections use readable expressions such as `size of nums`, `count 3 in nums`, and `position of "x" in words`.

GETTING STARTED

2.1 *Requirements*

To build the compiler from this repository, install:

- ◆ a C++17 compiler;
- ◆ CMake 3.20 or newer;
- ◆ Flex;
- ◆ Bison;
- ◆ LLVM with its C API;
- ◆ curl only when using the optional AI-assisted features.

The repository includes a root `CMakeLists.txt` and `build.py`. Paths to compilers and LLVM must be supplied by the local environment; LEXICO does not require machine-specific paths in source code.

2.2 *Build with CMake*

From the repository root:

```
cmake -S . -B build-cmake
cmake --build build-cmake --target lexico
```

The Python helper performs the same two CMake operations:

```
python build.py
```

The generated executable is normally placed under the selected CMake build directory's `bin` folder. The existing repository may also contain a previously built executable under `SourceCode/Applications`.

2.3 *Run a program*

LEXICO source files use the `.lx` extension:

```
lexico program.lx
```

The compiler accepts these options:

```
lexico --autofix program.lx
lexico --dump-expanded program.lx
```

- ◆ `--autofix` enables deterministic local repairs and optional AI parse-error recovery.
- ◆ `--dump-expanded` prints the source after load directives have been expanded.

2.4 *Your first complete program*

Create `hello.lx`:

```
create string name set value "LEXICO";
print "Hello,", name;
```

`create string name` declares a string variable. `set value` supplies its initial value. `print` accepts multiple expressions separated by commas; LEXICO prints a space between them and a newline at the end.

CHAPTER

3

LANGUAGE BASICS

3.1 *Statements*

Simple statements end with semicolons:

```
create int count set value 10;  
set count to count plus 1;  
print count;
```

The following constructs introduce blocks and therefore do not end their header with ;:

```
if count is greater than 0 then  
> print "positive";
```

3.2 *Comments*

A # begins a comment. The comment continues to the end of the line.

```
create int answer set value 42; # the answer
```

3.3 *Whitespace and block markers*

Spaces and tabs separate words. At the beginning of a line, consecutive > characters determine block depth:

```
if score is greater than 0 then
> print "outer block";
> if score is equal to 10 then
>> print "nested block";
print "outside";
```

One > enters the body of the nearest header. Two > characters enter a nested body. A line with no > returns to the outer level.

Blank lines do not add statements. Every non-empty statement inside a block still needs its normal semicolon.

3.4 *Identifiers and keywords*

Identifiers are names such as `total`, `user_name`, or `index2`. Keywords are lowercase and are reserved when used by the grammar. Examples include `create`, `set`, `print`, `if`, `while`, `routine`, `mesh`, `open`, and `file`.

Use descriptive names such as `item_count` and `average_price`. Avoid using language keywords as identifiers.

3.5 *Case sensitivity*

Keywords are lowercase. Write `create`, not `Create`; write `string`, not `String`.

```
create int value set value 4;
print value;
```

3.6 *Program structure*

A program is a sequence of top-level statements. Declarations, routines, classes, module loads, file blocks, and control-flow statements may appear according to their grammar rules. The implementation performs semantic checks for declarations, types, routine calls, collection contexts, and file contexts.

LEXICO does not provide a separate `main` function in the source language. The top-level source program is the program that executes.

PART

II

THE LANGUAGE



VARIABLES AND ASSIGNMENT



4.1 *Declare and initialize one variable*

```
create int age set value 21;  
create float temperature set value 21.5;  
create string city set value "Monastir";  
create bool online set value true;
```

The declared type controls the value accepted by semantic analysis and the way the value is printed or used.

4.2 *Declare several variables*

For multiple declarations, provide one value per identifier:

```
create int x, y, z set value 10, 20, 30;
```

The number of identifiers and values must match.

4.3 *Default initialization*

A declaration without `set value` receives a type-specific default:

```
create int count;  
create float ratio;  
create char initial;  
create string message;  
create bool ready;
```

The defaults are 0, 0.0, the null character, the empty string, and false.

4.4 *Assignment*

Use `set name to expression`; to update a scalar variable:

```
create int score set value 10;
set score to score plus 5;
print score;
```

The target must already be declared, and the assigned expression must be compatible with its type.

4.5 *Compound assignment shortcut*

A bare identifier followed by an arithmetic operator is shorthand for updating that same identifier:

```
create int counter set value 0;
counter plus 1;
counter times 2;
```

These are equivalent to:

```
set counter to counter plus 1;
set counter to counter times 2;
```

The full expression on the right follows normal precedence:

```
counter plus 3 times 4;
```

means `counter = counter + (3 * 4)`.

4.6 *Scope rules*

The language implementation uses a largely program-wide symbol model, with special context rules for routines, loops, classes, files, and collection-bound operations. A `for` iterator is implicit and intended for that loop. Do not rely on a block creating a fully independent lexical scope for ordinary declarations.

5

DATA TYPES AND LITERALS

5.1 *Type overview*

Type	Literal examples	Typical use
int	0, 42, -7	Whole-number calculations
float	3.14	Fractional values
char	'A', 'z'	One character
string	"hello"	Text
bool	true, false	Logical state

5.2 *Integers*

```
create int width set value 12;  
create int height set value 8;  
create int area set value width times height;  
print area;
```

Integer arithmetic supports plus, minus, times, divided by, and mod.

5.3 *Floating-point values*

```
create float price set value 12.50;  
create float tax set value 1.25;  
print price plus tax;
```

The compiler represents floats as double-precision values. Printed float formatting is handled by the runtime.

5.4 *Characters*

```
create char grade set value 'A';  
print grade;
```

A character literal contains one character. Character values can participate in text-oriented operations where the semantic rules permit conversion to or from strings.

5.5 *Strings*

```
create string first set value "Lex";  
create string second set value "ico";  
create string word set value first plus second;  
print word;
```

Strings support concatenation with plus, first-occurrence removal with minus, character indexing with at, length with length of, extraction expressions, and text input.

5.6 *Booleans*

```
create bool enabled set value true;  
set enabled to false;  
print enabled;
```

Booleans print as true or false. They are not numeric arithmetic operands. Use them in conditions and boolean meshes.

OPERATORS AND EXPRESSIONS



6.1 *Arithmetic operators*

Operator	Meaning
plus	Addition or string concatenation
minus	Subtraction or first substring removal
times	Multiplication
divided by	Division
mod	Integer remainder

```
create int a set value 17;  
create int b set value 5;  
print a plus b;  
print a minus b;  
print a times b;  
print a divided by b;  
print a mod b;
```

6.2 *Precedence*

Expressions are grouped in this order:

1. Parentheses;
2. times, divided by, mod;
3. plus, minus.

```
create int result set value 2 plus 3 times 4;
create int grouped set value (2 plus 3) times 4;
print result, grouped;
```

The first result is 14; the second is 20.

6.3 *Unary operators*

Unary minus negates numeric expressions. Unary plus is accepted as a positive-sign form.

```
create int debt set value -25;
set debt to plus 10;
print debt;
```

Applying arithmetic to booleans is invalid.

6.4 *Comparisons*

Conditions use these phrases:

- ◆ is equal to
- ◆ is not equal to
- ◆ is greater than
- ◆ is less than
- ◆ is greater or equal to
- ◆ is less or equal to

```
if temperature is greater than 30 then
> print "hot";
```

6.5 *Logical operators*

Conditions support not, and, and or:

```
if (age is greater or equal to 18) and not (blocked is equal to true)
  ⇨ then
> print "allowed";
```

not binds to the following condition atom. Parentheses make complex conditions clearer.

CHAPTER

7

STRINGS AND TEXT

7.1 *Concatenation*

```
create string first set value "Hello";  
create string second set value "LEXICO";  
print first plus " " plus second;
```

`plus` joins two strings. A character may be accepted in text-compatible contexts according to the semantic conversion rules.

7.2 *Remove a substring*

```
create string source set value "banana";  
create string result set value source minus "na";  
print result;
```

`minus` removes the first occurrence. If the substring does not occur, the original text remains unchanged.

7.3 *Index a string*

```
create string word set value "LEXICO";  
print word at 1;
```

Indexes are zero-based. The result is a character. An out-of-range index is a runtime error.

7.4 *Length*

```
create string title set value "LEXICO";  
print length of title;  
print length of "hello";
```

`length of` accepts string expressions. It is different from `size of`, which is used for collections.

7.5 *Extraction*

The grammar supports extracting text using a value from a source identifier:

```
create string text set value "one-two-three";  
print extract "two" from text;
```

Range extraction is also available:

```
print extract from 0 to 3 in text;
```

Use extraction with valid string sources and indexes/range expressions; invalid types are semantic errors.

INPUT AND OUTPUT

8.1 *Print values*

print accepts one or more expressions:

```
create int count set value 3;
print "count:", count, count plus 1;
```

Arguments are evaluated left-to-right, separated by spaces, followed by a newline.

8.2 *Read scalar input*

Input is part of declaration or assignment syntax:

```
create int age set value take user input with message "Age: ";
set age to take user input with message "Enter age again: ";
```

Supported scalar input is type-aware for integers, floats, characters, strings, and booleans.

8.3 *Message expressions*

A prompt can contain expressions separated by commas:

```
create int limit set value 10;
create int value set value take user input with message "Enter a value
↪ below ", limit, ": ";
```

A trailing comma is invalid. Message parts are printed before the input operation.

8.4 *Boolean input*

Boolean input is read as an integer-like value: 0 becomes `false`; a non-zero value becomes `true`.

CHAPTER

9

CONDITIONS

9.1 *Simple if*

```
create int score set value 75;  
  
if score is greater or equal to 50 then  
> print "Pass";
```

The body begins with > and contains normal semicolon-terminated statements.

9.2 otherwise

```
if score is greater or equal to 50 then  
> print "Pass";  
otherwise  
> print "Fail";
```

9.3 otherwise when

```
if score is greater or equal to 90 then
> print "A";
otherwise when score is greater or equal to 75 then
> print "B";
otherwise when score is greater or equal to 50 then
> print "C";
otherwise
> print "F";
```

Each otherwise when adds another condition. The final otherwise is optional.

9.4 *Nested conditions*

```
if account_ok is equal to true then
> if balance is greater than 0 then
>> print "transaction allowed";
> otherwise
>> print "insufficient balance";
```

The nested body uses two > markers.

10

LOOPS

10.1 while

Use while when the condition should be checked before each iteration.

```
create int i set value 0;  
while i is less than 3 do  
> print i;  
> set i to i plus 1;
```

The body may execute zero times.

10.2 *Numeric* for

```
for i from 1 to 5 do  
> print i;
```

The iterator is implicit and loop-local. Bounds are inclusive. An explicit step is optional:

```
for i from 0 to 10 step plus 2 do  
> print i;  
  
for i from 5 to 1 step minus 1 do  
> print i;
```

The step must be integer-compatible and must not be zero.

10.3 *Collection-bound for*

```
create int mesh nums set values 10, 20, 30;
for i in nums from 0 to size of nums minus 1 do
> print nums at i;
```

The collection name after `in` binds the loop to a mesh. Contextual collection operations can use the current element position.

10.4 *repeat until*

Use `repeat` when the body must run before the condition is checked:

```
create int attempts set value 0;
repeat
> set attempts to attempts plus 1;
> print attempts;
until attempts is greater or equal to 3 then
```

The body runs at least once.

10.5 *attempt up to*

Use `attempt` for bounded retries:

```
create int tries set value 0;
create int value set value 0;

attempt up to 3 while value is less than 10 do
> set value to value plus 3;
> set tries to tries plus 1;

on failure
> print "The limit was reached after", tries, "tries";
```

The alternative spelling `times` while is also tokenized:

```
attempt up to 3 times while value is less than 10 do
> set value to value plus 3;
```

The failure block is optional and runs only when the attempt limit is reached while the condition remains true.

10.6 *Matrix loops*

Matrices have a two-iterator form:

```
create int matrix mesh grid rows 2 cols 3;  
for i j in grid i from 0 to 1 and j from 0 to 2 do  
> set i plus j at i,j;
```

The two iterators represent row and column coordinates and are implicit.

PART

III

BUILDING PROGRAMS



11

ROUTINES

11.1 *Define a routine*

A global routine has a name, optional typed parameters, a return type, and a block:

```
define routine add with args in take int left, int right returns int
  ↪ do
  > giveback left plus right;
```

`giveback` returns the expression. The returned type must match the declared return type.

11.2 *Call a routine and use its result*

```
create int total set value outcome of add with args in take 4, 6;
print total;
```

`outcome of` is an expression and can appear in declarations, assignments, print arguments, and larger expressions.

11.3 *Discard a result with* `emit`

```
emit add with args in take 1, 2;
```

`emit` calls a routine for its side effects and discards its result.

11.4 No-argument calls

```
define routine answer returns int do
> giveback 42;

print outcome of answer;
emit answer;
```

11.5 Parameters and validation

```
define routine describe with args in take string name, int age returns
  ↪ string do
> giveback name plus " is " plus age;

print outcome of describe with args in take "Ada", 36;
```

The compiler checks:

- ♦ routine existence;
- ♦ argument count;
- ♦ argument types;
- ♦ return type compatibility;
- ♦ giveback placement and type.

Global routine definitions use `define routine`. The language does not accept `define function` as an alternative spelling.

11.6 Recursion

The grammar permits a routine to call another routine through `outcome of` or `emit`. Recursive programs should still obey the normal declaration, return-type, and runtime limits. Example shape:

```
define routine countdown with args in take int n returns int do
> if n is less or equal to 0 then
>> giveback 0;
> giveback outcome of countdown with args in take n minus 1;
```

Use recursion only when the routine's declared return type and all paths are valid for the current semantic analyzer.

12

MESHES, ARRAYS, TABLES, AND MATRICES

LEXICO exposes collection features through the keywords `array`, `table`, and the preferred readable `mesh` forms.

12.1 *Typed mesh with a size*

```
create int mesh nums sized 5;
```

A typed mesh has one element type. The size expression must be integer-compatible.

12.2 *Dynamic mesh*

```
create mesh mixed sized 4;
```

A dynamic mesh/table can contain mixed scalar values. Values retain run-time type tags.

12.3 *Initialize from values*

```
create int mesh nums set values 12, 32, 12, 54, 25;  
create string mesh words set values "alpha", "beta", "gamma";  
create mesh mixed set values 12, "hello", 'x', 3.14, true;
```

Typed collections enforce compatible values. Dynamic collections accept mixed scalar values.

12.4 *Legacy explicit array/table forms*

The grammar also accepts:

```
create array of int values with size 10;
create table rows with size 5;
```

The mesh vocabulary is the recommended user-facing style.

12.5 *Read and write elements*

```
create int mesh nums set values 10, 20, 30;
print nums at 1;
set nums value at 1 to 25;
print nums at 1;
```

Indexes are zero-based. Out-of-range access is a runtime error.

Inside a collection-bound loop, contextual assignment is available:

```
for i in nums from 0 to size of nums minus 1 do
> set value to nums at i plus 1;
```

12.6 *Append and sort*

```
append 40 to nums;
append 5 to nums at 0;
sort nums ascendantly;
sort nums descendantly;
```

Sorting is supported for typed collections of supported scalar types. Dynamic collections cannot use typed ordering safely.

12.7 *Count, membership, and position*

```
print count 25 in nums;
print check 25 in nums;
print position of 25 in nums;
```

- ◆ count value in collection returns an integer count;
- ◆ check value in collection returns a boolean;
- ◆ position of value in collection returns the first index or -1.

For dynamic collections, runtime type tags matter: 1, '1', and "1" are distinct values.

12.8 *Matrices*

Declare a fixed-size typed matrix:

```
create int matrix mesh grid rows 2 cols 3;
```

Initialize rows using /:

```
create int matrix mesh grid set values 1, 2, 3 / 4, 5, 6;
```

All initializer rows must have equal lengths.

Read and write cells:

```
print grid at 0,1;  
set grid at 0,1 to 99;
```

Matrix dimensions:

```
print size of grid;  
print row length of grid;  
print column length of grid;
```

Matrices support row-major append, count/check/position expressions, and coordinate loops.

FILE OPERATIONS

File operations are statements inside an active file block.

13.1 *Open or create a file*

```
open "notes.txt" file make do  
> write "first line" in file;  
> write "second line" in file;
```

`file do` requires an existing file. `file make do` creates it if necessary. The active file is closed automatically when the block finishes.

13.2 *Write lines*

Append a line:

```
write "new line" in file;
```

Replace or create a particular zero-based line:

```
write "updated" in line 1 in file;
```

13.3 *Read lines*

Read all content into a table target:

```
create table rows with size 0;  
read file into rows;
```

Read one line into a string:

```
create string line set value "";
read line 0 in file into line;
print line;
```

Read a prefix or range:

```
read 3 lines in file into rows;
read 2 lines from line 1 in file into rows;
```

Single-line reads require a string scalar. Multi-line reads require a table target.

13.4 *Clear, rename, and close*

```
clear line 0 in file;
clear from line 1 to line 2 in file;
set file title to "archive.txt";
close file;
```

These operations are only valid while a file block is active. File indexes and counts must be integer expressions.

13.5 *Complete file example*

```
create string line set value "";
create table rows with size 0;

open "notes.txt" file make do
> clear file;
> write "alpha" in file;
> write "beta" in file;
> read line 1 in file into line;
> print "Read:", line;
> read file into rows;
> print rows;
```

CHAPTER

14

MODULES

Use `load` to include another `.lx` file:

```
load "lib/crypto.lx";
```

Relative paths are resolved from the file containing the `load` statement. Loading is recursive. Duplicate loads are ignored and circular loads are rejected.

Place reusable routines or classes in library files and application logic in a top-level file:

```
load "lib/routines.lx";  
load "lib/classes.lx";  
  
create int result set value outcome of add with args in take 2, 3;  
print result;
```

CLASSES AND PROFILES

LEXICO supports a class-oriented model with properties, profiles, inheritance, routines, and overrides.

15.1 *Define a class*

```
define class Shape include
> create string label;
> profile base do
>> set label to "shape";
> routine show do
>> print label;
```

A class property is declared with `create type name;`. A profile supplies initialization actions for the properties. A class routine may omit returns; its default routine return type is the implementation's default routine type, and it is normally used with `emit`.

15.2 *Inheritance and override*

```
define class Circle inherits Shape include
> override create string label;
> create int radius;
> profile default do
>> set label to "circle";
>> set radius to 2;
> override routine show do
>> print label, radius;
```

`override create` replaces an inherited property. `override routine` replaces an inherited routine. The parent member must exist.

15.3 *Instantiate a profile*

```
create Circle c using default;  
emit c_show;
```

Class instantiation expands profile actions and creates accessible member names for the generated object. In the current implementation, member references are represented internally with object-prefixed names; programmers should use the class/profile forms shown in the language guide and examples.

15.4 *Class routine with a return type*

```
define class Counter include  
> create int value;  
> profile start do  
>> set value to 0;  
> routine get returns int do  
>> giveback value;
```

Use the generated callable according to the class instantiation model and the routine's declared return type.

CONVERSIONS AND BUILT-IN EXPRESSIONS

16.1 *Statement conversion*

```
create int code set value 65;
convert code to char;
convert code to string;
```

Statement conversion changes a scalar variable in place. Supported destination types are int, float, char, and string. Conversion is checked semantically and may fail at runtime for invalid string input.

16.2 *Expression conversion*

```
create string text set value "42";
create int number set value convert text to int;
print number;
```

The expression form is `convert expression to type`.

16.3 *Built-in expressions*

Expression	Result
size of mesh_name	Collection size
row length of matrix_name	Matrix row count
column length of matrix_name	Matrix column count
length of string_expression	String length
count value in collection	Number of matches
check value in collection	Boolean membership

Expression	Result
position of value in collection	First matching index or -1
collection at index	Element at a zero-based index
matrix at row,col	Matrix cell
outcome of routine ...	Routine return value

PART

IV

PRACTICE



ERRORS AND LIMITATIONS

LEXICO reports errors in stages from the programmer's perspective:

1. lexical errors for invalid or oversized tokens;
2. parse errors for invalid structure or missing syntax;
3. semantic errors for undeclared names, incompatible types, invalid contexts, or incorrect routine calls;
4. runtime errors for invalid indexes, failed conversions, and file failures.

There is no general try/catch language construct in the current grammar. The attempt ... on failure construct is a bounded retry loop, not exception handling.

Common mistakes:

<pre>Create int value set value 1; create int value set value 1 if value > 0 then > print value;</pre>	<pre># keywords must be lowercase # missing semicolon # use the word comparison form</pre>
--	--

Correct forms:

```
create int value set value 1;
if value is greater than 0 then
> print value;
```

Other important limitations:

- ◆ ordinary declarations use the implementation's largely program-wide symbol model;
- ◆ loop iterators are implicit and intended for their loop;
- ◆ file operations require an active file block;
- ◆ typed collections enforce element compatibility;
- ◆ step 0 is invalid;

- ◆ collection and matrix indexes are zero-based;
- ◆ the current AI feature is optional and does not replace deterministic validation.

COMPLETE PROGRAMS

*18.1 Beginner: total calculator*

```
create int price set value 12;
create int quantity set value 4;
create int total set value price times quantity;

print "Price:", price;
print "Quantity:", quantity;
print "Total:", total;
```

This demonstrates declarations, arithmetic, and multi-expression output.

18.2 Intermediate: grade classification

```
create int score set value 82;

if score is greater or equal to 90 then
> print "A";
otherwise when score is greater or equal to 75 then
> print "B";
otherwise when score is greater or equal to 50 then
> print "C";
otherwise
> print "F";
```

The program checks conditions from top to bottom and executes the first matching branch.

18.3 *Intermediate: routine and loop*

```
define routine square with args in take int value returns int do
> giveback value times value;

create int i set value 1;
while i is less or equal to 5 do
> print "square", i, "=", outcome of square with args in take i;
> set i to i plus 1;
```

The routine returns a value; the loop invokes it once per iteration.

18.4 *Advanced: mesh search*

```
create int mesh scores set values 12, 45, 78, 45, 91;
create int target set value 45;

print "count:", count target in scores;
print "present:", check target in scores;
print "first index:", position of target in scores;

for i in scores from 0 to size of scores minus 1 do
> if scores at i is greater than 50 then
>> print "high score at", i, scores at i;
```

This combines typed mesh initialization, membership expressions, indexing, a collection loop, and nested conditions.

18.5 *Advanced: file-backed notes*

```
create string selected set value "";
create table all_lines with size 0;

open "notes.txt" file make do
> clear file;
> write "learn declarations" in file;
> write "learn loops" in file;
> read line 1 in file into selected;
> print "Selected:", selected;
> read file into all_lines;
> print all_lines;
```

The file block creates a file, writes lines, reads one string line, reads all lines into a table, and prints the table.

CHAPTER

19

RECIPES

19.1 *Factorial with a loop*

```
create int n set value 5;
create int result set value 1;
create int i set value 1;

while i is less or equal to n do
> set result to result times i;
> set i to i plus 1;

print "factorial:", result;
```

19.2 *Find a maximum in a mesh*

```
create int mesh values set values 7, 2, 19, 4, 11;
create int maximum set value values at 0;

for i in values from 1 to size of values minus 1 do
> if values at i is greater than maximum then
>> set maximum to values at i;

print "maximum:", maximum;
```

19.3 *Sort a typed mesh*

```
create int mesh values set values 7, 2, 19, 4, 11;
sort values ascendantly;
print values;
```

19.4 *Validate a bounded retry*

```
create int attempts set value 0;
create int answer set value 0;

attempt up to 3 while answer is not equal to 7 do
> set answer to take user input with message "Enter 7: ";
> set attempts to attempts plus 1;

on failure
> print "No correct answer after", attempts, "attempts";
```

19.5 *Convert text to a number*

```
create string text set value "100";
create int number set value convert text to int;
print number plus 1;
```

Invalid numeric text produces a runtime conversion error.

PART

V

REFERENCE



LANGUAGE REFERENCE

20.1 *Keywords*

create, set, value, values, print, convert, append, sort, ascendantly, descendantly, count, check, position, extract, if, then, otherwise, when, while, do, for, from, to, step, repeat, until, attempt, up to, times while, on failure, define, routine, returns, giveback, emit, outcome, of, with, args, in, take, user, input, message, class, include, inherits, profile, using, override, open, make, file, write, read, line, lines, clear, close, title, into, load, array, table, mesh, matrix, rows, cols, row, column, size, length, sized, at, true, false, int, float, char, string, bool.

20.2 *Statement forms*

```
create <type> <name> set value <expression>;
create <type> <name>;
create <type> <name>, <name> set value <expr>, <expr>;
set <name> to <expression>;
set <name> to take user input [with message <parts>];
<name> plus|minus|times|divided by|mod <expression>;
print <expression>[, <expression>...];
convert <name> to <type>;
```

20.3 *Control-flow forms*

```
if <condition> then
> <statements>
otherwise [when <condition> then]

while <condition> do
> <statements>
```

```

for <i> from <start> to <end> [step <step>] do
> <statements>

for <i> in <mesh> from <start> to <end> [step <step>] do
> <statements>

repeat
> <statements>
until <condition> then

attempt up to <n> while <condition> do
> <statements>
on failure
> <statements>

```

20.4 *Routine forms*

```

define routine <name> [with args in take <type> <parameter>, ...]
returns <type> do
> <statements>

giveback <expression>;
emit <name> [with args in take <arguments>];
outcome of <name> [with args in take <arguments>]

```

20.5 *Collection forms*

```

create <type> mesh <name> sized <expression>;
create <type> mesh <name> set values <values>;
create mesh <name> sized <expression>;
create mesh <name> set values <values>;
print <name> at <index>;
set <name> value at <index> to <expression>;
append <expression> to <name>;
append <expression> to <name> at <index>;
sort <name> ascendantly;
count <expression> in <name>;
check <expression> in <name>;
position of <expression> in <name>;

```

20.6 *File forms*

```
open <string-expression> file do
> <file statements>

open <string-expression> file make do
> <file statements>

write <string> in file;
write <string> in line <int> in file;
read file into <target>;
read line <int> in file into <string-target>;
read <int> lines in file into <table-target>;
read <int> lines from line <int> in file into <table-target>;
clear file;
clear line <int> in file;
clear from line <int> to line <int> in file;
set file title to <string>;
close file;
```

CHAPTER

21

CHEAT SHEET

```
# Variables
create int count set value 0;
create string name set value "LEXICO";
create bool ready set value true;

# Assignment and expressions
set count to count plus 1;
count plus 1;
set average to (a plus b) divided by 2;

# Output and input
print "count:", count;
create int n set value take user input with message "n: ";

# Conditions
if count is greater than 0 then
> print "positive";
otherwise
> print "zero or negative";

# Loops
while count is less than 5 do
> set count to count plus 1;

for i from 0 to 3 do
> print i;

repeat
> set count to count plus 1;
until count is greater than 10 then

# Routines
define routine double with args in take int x returns int do
> giveback x times 2;
```

```
print outcome of double with args in take 4;

# Meshes
create int mesh nums set values 3, 1, 2;
print check 2 in nums;
sort nums ascendantly;

# Files
open "notes.txt" file make do
> write "hello" in file;

# Modules
load "library.lx";
```

LEXICO programs are easiest to read when each statement has one clear purpose, block markers are aligned consistently, and expressions use parentheses when the grouping matters.